Notatnik w Tkinterze – notatka na sprawdzian

Program to prosta aplikacja okienkowa „Notes app”. Pozwala dodawać notatki (tytuł + treść), usuwać je, edytować i podglądać (podwójne kliknięcie). Napisany jest w Pythonie z biblioteką **tkinter** (graficzny interfejs użytkownika, czyli GUI).

Spis treści

1. Część A – słowniczek: co robi każdy element (Python)
2. Część B – słowniczek: co robi każdy element (tkinter)
3. Część C1 – cały kod z komentarzami do każdej linijki
4. Część C2 – kod blok po bloku z objaśnieniem
5. Część D – pułapki i błędy w programie
6. Część E – ściąga: pack / grid / place
7. Część F – przykładowe pytania (zamknięte i otwarte) z odpowiedziami

---

Część A – Python: co do czego służy

Importowanie

Element	Do czego służy	Przykład z kodu
`import modul`	Wczytuje cały moduł (bibliotekę).	`import tkinter`
`import modul as skrot`	Wczytuje moduł pod krótszą nazwą (aliasem). Potem piszemy `tk.` zamiast `tkinter.`.	`import tkinter as tk`
`from modul import nazwa`	Wczytuje z modułu tylko wybrane elementy. Używamy ich bez prefiksu.	`from tkinter import ttk, messagebox`
`from datetime import datetime`	Z modułu `datetime` bierzemy klasę `datetime` (nazywają się tak samo).	`datetime.now()`

Klasy i obiekty

Element	Do czego służy
`class Note:`	Definiuje **klasę**, czyli „przepis” na obiekty. Tu: przepis na notatkę.
obiekt (instancja)	Konkretny egzemplarz klasy, np. `Note("Zakupy", "mleko")`.
`__init__`	**Konstruktor**, czyli specjalna metoda wywoływana automatycznie przy tworzeniu obiektu. Ustawia początkowe wartości atrybutów. Dwa podkreślenia z każdej strony („dunder”).
`self`	Pierwszy parametr metody, oznacza „ten konkretny obiekt”. Przez `self.cos = ...` zapisujemy dane w obiekcie.

Element	Do czego służy
`self.title`, `self.content`, `self.date`	**Atrybuty** obiektu, czyli zmienne należące do obiektu.
`title=""`	**Wartość domyślna parametru**. Jeśli nie podamy tytułu, będzie pusty napis. Można więc napisać `Note()`.
`notes[0].title`	Dostęp do atrybutu obiektu (kropka).

Data i czas

Element	Do czego służy
`datetime.now()`	Zwraca aktualną datę i godzinę.
`.strftime("%d/%m/%Y %H:%M:%S")`	Zamienia datę na napis w podanym formacie („string format time”).
`%d`	dzień (01–31)
`%m`	**miesiąc** (01–12), małe m
`%Y`	rok czterocyfrowy (2026)
`%H`	godzina 00–23
`%M`	**minuty**, wielkie M (nie myl z `%m`!)
`%S`	sekundy

Wynik to np. `"29/10/2026 21:43:05"`.

Typy danych i operacje

Element	Do czego służy	Przykład
krotka (tuple) `( )`	Niezmienna sekwencja wartości. W tkinterze np. czcionka.	`("Helvetica", 9, "bold")`
lista `[ ]`	Zmienna sekwencja, można dodawać i usuwać.	`notes = []`
`lista.append(x)`	Dodaje element **na koniec** listy.	`notes.append(Note(...))`
`lista.insert(i, x)`	Wstawia element na pozycję `i` (0 = na początek).	`notes.insert(0, Note(...))`
`lista.pop()`	Usuwa i zwraca **ostatni** element. Na pustej liście: błąd `IndexError`.	`notes.pop()`
`lista.pop(i)`	Usuwa element o indeksie `i`.	`notes.pop(selected[0])`
`lista.clear()`	Usuwa wszystkie elementy.	`notes.clear()`
`lista[i]`	Dostęp do elementu po indeksie (od 0).	`notes[selected[0]]`
`napis.strip()`	Usuwa białe znaki (spacje, tabulatory, entery) z początku i końca napisu.	`" abc ".strip()` daje `"abc"`

Sterowanie i funkcje

Element	Do czego służy
`def nazwa():`	Definiuje funkcję.
`def show_content(event):`	Funkcja z parametrem. Tkinter sam przekazuje obiekt zdarzenia przy `bind`.
`if warunek:`	Instrukcja warunkowa.
`if title:`	Sprawdza, czy napis **nie jest pusty**. Pusty napis `""`, pusta krotka `()`, `0`, `None` i pusta lista są „fałszywe” (falsy).
`for note in notes:`	Pętla po każdym elemencie listy.
zmienna globalna	`notes`, `button_font` są zdefiniowane poza funkcjami, więc widać je wszędzie. Funkcje mogą **modyfikować** listę `notes` (append, pop) bez słowa `global`, bo nie przypisują jej na nowo (`notes = ...`).

Część B – tkinter: co do czego służy

Podstawy

Element	Do czego służy
`tkinter` (`tk`)	Standardowa biblioteka Pythona do tworzenia okienek (GUI).
`ttk`	„Themed tkinter”: nowocześniej wyglądające wersje widżetów (`ttk.Label`, `ttk.Entry`, `ttk.Button`...).
`messagebox`	Gotowe okienka dialogowe: `showinfo`, `showwarning`, `showerror`, `askyesno`...
widżet	Każdy element okna: etykieta, przycisk, pole tekstowe, lista, ramka.
rodzic (master)	Pierwszy argument widżetu mówi, w czym on leży, np. `tk.Button(button_frame, ...)` to przycisk w ramce.
`tk.END`	Stała oznaczająca „koniec” (np. koniec listy lub tekstu). Jej wartość to napis `"end"`.

Okno główne

Element	Do czego służy
`window = tk.Tk()`	Tworzy **główne okno** aplikacji. Musi być tylko jedno.
`window.title("Notes app")`	Ustawia tekst na pasku tytułu.
`window.geometry('350x550')`	Rozmiar okna: **szerokość x wysokość** w pikselach. Można dodać pozycję: `'350x550+100+50'`.
`window.configure(background="#2C2C2C")`	Zmienia opcje widżetu po jego utworzeniu (tu kolor tła). `config` to to samo co `configure`.
`window.mainloop()`	Uruchamia **pętlę zdarzeń**: okno się pokazuje i czeka na kliknięcia i klawisze. Kod po `mainloop()` wykona się dopiero po zamknięciu okna. Bez tego okno się nie pokaże (albo od razu zniknie).

Widżety użyte w programie

Widżet	Do czego służy
`ttk.Label` / `tk.Label`	**Etykieta**: wyświetla tekst (nie da się go edytować).
`ttk.Entry`	**Pole tekstowe** w jednej linii.
`tk.Listbox`	**Lista elementów**, z której można zaznaczać pozycje.
`tk.Frame`	**Ramka**: niewidoczny (lub kolorowy) kontener grupujący inne widżety.
`tk.Button`	**Przycisk**. Po kliknięciu wywołuje funkcję z `command`.

Najczęstsze opcje widżetów

Opcja	Znaczenie
`text="..."`	Tekst wyświetlany na widżecie.
`font=("Arial", 20, "bold")`	Czcionka: (rodzina, rozmiar, styl). Styl: `"bold"`, `"italic"`, `"underline"`, `"normal"`.
`background` / `bg`	Kolor tła (`bg` to skrót).
`foreground` / `fg`	Kolor tekstu (`fg` to skrót).
Kolory	Nazwy (`"orange red"`, `"red3"`, `"lightgreen"`, `"cyan2"`, `"white"`) albo hex `"#RRGGBB"` (np. `"#2C2C2C"` to ciemnoszary).
`borderwidth=0`	Grubość obramowania (0 = brak).
`highlightthickness=0`	Usuwa ramkę podświetlenia pokazywaną, gdy widżet ma fokus.
`command=funkcja`	Funkcja wywoływana po kliknięciu przycisku. **Bez nawiasów!** `command=add_last`, a nie `command=add_last()`.

Metody pola Entry

Metoda	Do czego służy
`entry.get()`	Zwraca wpisany tekst (napis).
`entry.delete(0, tk.END)`	Usuwa tekst od znaku 0 do końca, czyli czyści pole.
`entry.insert(0, "tekst")`	Wstawia tekst na pozycję 0 (na początek).

Metody Listboxa

Metoda	Do czego służy
`listbox.insert(tk.END, "x")`	Dodaje element na koniec listy.
`listbox.insert(0, "x")`	Dodaje element na początek.
`listbox.delete(0, tk.END)`	Usuwa wszystkie elementy.
`listbox.delete(tk.END, tk.END)` / `delete(tk.END)`	Usuwa ostatni element.
`listbox.delete(i)`	Usuwa element o indeksie `i`.

Metoda	Do czego służy
`listbox.curselection()`	Zwraca **krotkę indeksów** zaznaczonych elementów, np. `(2,)`. Gdy nic nie jest zaznaczone: pustą krotkę `()`. Dlatego używamy `selected[0]`.
`listbox.select_set(i)`	Zaznacza element o indeksie `i` (programowo).

Menedżery geometrii (rozmieszczanie widżetów)

Widżet **nie pojawi się w oknie**, dopóki nie wywołamy na nim `pack()`, `grid()` albo `place()`.

Metoda	Jak działa	Opcje użyte w kodzie
`pack()`	Układa widżety **jeden pod drugim** (domyślnie od góry).	`fill="x"`: rozciągnij w poziomie. `padx=20`: margines zewnętrzny lewo/prawo. `pady=5`: margines góra/dół. `pady=(0,5)`: góra 0, dół 5. `anchor="w"`: przyklej do lewej (w = west).
`pack_forget()`	**Ukrywa** widżet spakowany przez `pack` (widżet dalej istnieje, można go znów spakować).	—
`grid()`	Układa widżety w **tabeli** (wiersze i kolumny).	`row=0, column=1`: komórka. `sticky="ew"`: rozciągnij do wschodu i zachodu (lewo–prawo). `padx`, `pady`: marginesy.
`grid_columnconfigure(nr, weight=1, uniform="grupa")`	Konfiguruje kolumnę siatki.	`weight=1`: kolumna dostaje część wolnego miejsca (rośnie z oknem). `uniform="buttons"`: kolumny z tej samej grupy mają **równą szerokość**.
`place()`	Umieszcza widżet w **konkretnym miejscu**.	`relx=0.5, rely=0.45`: pozycja względna (0–1, czyli 50% szerokości, 45% wysokości). `anchor="center"`: punktem zaczepienia jest środek widżetu.

Zasada: w jednym kontenerze (oknie albo ramce) **nie mieszamy `pack` i `grid`**. W programie okno używa `pack` (i `place` dla dymka), a ramki z przyciskami używają `grid` wewnątrz siebie. To jest poprawne.

Kierunki w tkinterze: `n` (góra), `s` (dół), `e` (prawo), `w` (lewo), `center`.

Zdarzenia i timery

Element	Do czego służy
`widget.bind("<Double-Button-1>", funkcja)`	Podpina funkcję pod **zdarzenie**. `<Double-Button-1>` to podwójne kliknięcie lewym przyciskiem myszy. Inne: `<Button-1>` (kliknięcie), `<Button-3>` (prawy), `<Return>` (Enter), `<Key>` (dowolny klawisz).

Element	Do czego służy
`event`	Obiekt zdarzenia przekazywany automatycznie do funkcji z `bind` (zawiera np. `event.x`, `event.y`, `event.widget`). Dlatego `show_content(event)` musi mieć parametr. Funkcje z `command=` nie dostają argumentu.
`widget.after(3000, funkcja)`	Wywołuje funkcję **po 3000 ms** (3 s), bez zatrzymywania programu.
`widget.destroy()`	**Niszczy** widżet (usuwa go całkowicie).
`messagebox.showinfo(tytul, wiadomosc, detail=...)`	Okienko informacyjne. `detail`: dodatkowy, mniejszy tekst pod wiadomością (tu data).

`pack_forget()` czy `destroy()`?

- `pack_forget()`: **ukrywa**, widżet istnieje, można go pokazać przez `pack()`.
- `destroy()`: **usuwa na zawsze**.

`tk` czy `ttk`?

- `tk.Label`, `tk.Button`: klasyczne, łatwo ustawić `bg` i `fg`.
- `ttk.Label`, `ttk.Entry`: ładniejsze (motywy systemowe); część opcji wyglądu ustawia się przez style (`ttk.Style`). `ttk.Label` przyjmuje jednak `background`, `foreground` i `font`.

Część C1 – cały kod z komentarzami do każdej linijki

Poniżej cały program z komentarzem nad każdą linijką: co to jest, gdzie leży (rodzic), jaki ma kolor, czcionkę, padding x i y, w którym wierszu i kolumnie siatki jest i co robi. Kod działa identycznie jak oryginał. Ta sama wersja jest w osobnym pliku `notatnik_tkinter_komentarze.py`.

```
# ======================================================================
#  NOTATNIK W TKINTERZE – WERSJA Z KOMENTARZAMI DO KAŻDEJ LINIJKI
#  Program działa identycznie jak oryginał, dodane są tylko komentarze.
# ======================================================================


# ----------------------------------------------------------------------
#  IMPORTY
# ----------------------------------------------------------------------

# Importujemy bibliotekę tkinter (okienka / GUI) pod skrótem "tk".
# Dzięki "as tk" piszemy np. tk.Button zamiast tkinter.Button.
import tkinter as tk

# Z pakietu tkinter importujemy dwa moduły:
#   ttk        – ładniejsze ("themed") wersje widżetów, np. ttk.Label, ttk.Entry
#   messagebox – gotowe okienka dialogowe, np. messagebox.showinfo(...)
from tkinter import ttk, messagebox

# Z modułu datetime importujemy klasę datetime – posłuży do pobrania
```

```
# aktualnej daty i godziny (datetime.now()).
from datetime import datetime


# ---------------------------------------------------------------------
#  KLASA NOTE – "przepis" na jedną notatkę
# ---------------------------------------------------------------------

# Definicja klasy o nazwie Note (notatka).
class Note:
    # __init__ = KONSTRUKTOR. Wywołuje się automatycznie, gdy tworzymy
    # obiekt, np. Note("Zakupy", "mleko").
    #   self    – ten konkretny tworzony obiekt
    #   title   – tytuł notatki, wartość domyślna "" (pusty napis)
    #   content – treść notatki, wartość domyślna "" (pusty napis)
    def __init__(self, title="", content=""):
     # Zapisujemy przekazany tytuł w atrybucie obiektu "title".
     self.title = title
     # Zapisujemy przekazaną treść w atrybucie obiektu "content".
     self.content = content
     # Zapisujemy datę utworzenia notatki jako napis.
     #   datetime.now()  – aktualna data i godzina
     #   .strftime(...)  – zamiana daty na tekst w podanym formacie:
     #       %d – dzień (01-31)      %m – miesiąc (01-12)
     #       %Y – rok 4-cyfrowy      %H – godzina (00-23)
     #       %M – minuty (00-59)     %S – sekundy (00-59)
     #   Wynik np.: "29/10/2026 21:43:05"
     self.date = datetime.now().strftime("%d/%m/%Y %H:%M:%S")


# ---------------------------------------------------------------------
#  ZMIENNE GLOBALNE (widoczne w całym programie)
# ---------------------------------------------------------------------

# Czcionka używana na wszystkich przyciskach – KROTKA (tuple) z 3 elementami:
#   "Helvetica" – rodzina (krój) czcionki
#   9           – rozmiar czcionki
#   "bold"      – styl: pogrubiona
button_font = ("Helvetica", 9, "bold")

# Pusta LISTA, w której będziemy trzymać obiekty Note (dane programu).
notes = []

# Na start dodajemy jedną przykładową notatkę na KONIEC listy (append):
#   tytuł  = "Pierwsza notatka"
#   treść  = "Treść pierwszej notatki"
notes.append(Note("Pierwsza notatka", "Treść pierwszej notatki"))


# ---------------------------------------------------------------------
#  FUNKCJE PODPIĘTE POD PRZYCISKI
#  (każda zmienia DWA miejsca: listę "notes" z danymi
#   oraz Listbox "note_list", który widać na ekranie)
# ---------------------------------------------------------------------
```

```
# Funkcja dla przycisku "Add last" – dodaje notatkę NA KONIEC.
def add_last():
    # Pobieramy tekst z pola "Title" (.get()) i usuwamy spacje
    # z początku i końca (.strip()).
    title = note_title_entry.get().strip()
    # Jeśli tytuł NIE jest pusty (pusty napis "" traktowany jest jak False)...
    if title:
     # ...pobieramy tekst z pola "Content".
     content = note_content_entry.get()
     # Tworzymy nowy obiekt Note i dodajemy go na KONIEC listy notes.
     notes.append(Note(title, content))
     # Dodajemy tytuł na KONIEC listy na ekranie (tk.END = "koniec").
     note_list.insert(tk.END, title)


# Funkcja dla przycisku "Add first" – dodaje notatkę NA POCZĄTEK.
def add_first():
    # Pobieramy tytuł z pola "Title" bez spacji na brzegach.
    title = note_title_entry.get().strip()
    # Jeśli tytuł nie jest pusty...
    if title:
     # ...pobieramy treść z pola "Content".
     content = note_content_entry.get()
     # Wstawiamy nowy obiekt Note na pozycję 0 (czyli na początek listy).
     notes.insert(0, Note(title, content))
     # Wstawiamy tytuł na pozycję 0 w Listboxie (na samą górę).
     note_list.insert(0, title)


# Funkcja dla przycisku "Delete last" – usuwa OSTATNIĄ notatkę.
def delete_last():
    # Usuwamy ostatni element z listy danych.
    # UWAGA: gdy lista jest pusta -> błąd IndexError (brak sprawdzenia if).
    notes.pop()
    # Usuwamy ostatni element z Listboxa (od końca do końca).
    note_list.delete(tk.END, tk.END)


# Funkcja dla przycisku "Delete all" – usuwa WSZYSTKO i pokazuje dymek.
def delete_all():
    # Czyścimy całą listę danych.
    notes.clear()
    # Usuwamy z Listboxa elementy od indeksu 0 do końca (czyli wszystkie).
    note_list.delete(0, tk.END)
    # Tworzymy etykietę (tk.Label) z napisem "Deleted":
    #   brak rodzica         -> trafia do głównego okna
    #   background="black"   -> czarne tło
    #   foreground="white"   -> biały tekst
    #   font=button_font     -> czcionka Helvetica, 9, pogrubiona
    #   text="Deleted"       -> wyświetlany tekst
    label_toast = tk.Label(background="black", foreground="white", font=button_font,
text="Deleted")
    # Umieszczamy etykietę metodą place (dokładna pozycja):
```

```
    #   relx=0.5         -> 50% szerokości okna (środek w poziomie)
    #   rely=0.45        -> 45% wysokości okna
    #   anchor="center"  -> w tym punkcie leży ŚRODEK etykiety
    label_toast.place(relx=0.5, rely=0.45, anchor="center")
    # Po 3000 milisekundach (3 sekundach) wywołujemy label_toast.destroy,
    # czyli usuwamy etykietę. destroy BEZ nawiasów – przekazujemy funkcję,
    # a nie wywołujemy jej od razu. after() nie blokuje programu.
    label_toast.after(3000, label_toast.destroy)


# Funkcja dla przycisku "Delete selected" – usuwa ZAZNACZONĄ notatkę.
def delete_selected_item():
    # curselection() zwraca KROTKĘ indeksów zaznaczonych elementów,
    # np. (2,). Gdy nic nie jest zaznaczone -> pusta krotka ().
    selected = note_list.curselection()
    # Jeśli coś jest zaznaczone (pusta krotka = False)...
    if selected:
     # ...usuwamy z listy danych element o tym indeksie (selected[0]).
     notes.pop(selected[0])
     # ...usuwamy ten element z Listboxa.
     note_list.delete(selected)


# Funkcja wywoływana po PODWÓJNYM KLIKNIĘCIU na notatkę (przez bind).
# Ma parametr "event", bo bind automatycznie przekazuje obiekt zdarzenia.
def show_content(event):
    # Pobieramy krotkę z indeksem zaznaczonej notatki.
    selected = note_list.curselection()
    # Pokazujemy okienko informacyjne:
    #   1. argument -> tytuł okienka   = tytuł notatki
    #   2. argument -> główny tekst     = treść notatki
    #   detail=     -> dodatkowy tekst  = data utworzenia
    # UWAGA: gdy nic nie jest zaznaczone -> błąd IndexError.
    messagebox.showinfo(notes[selected[0]].title, notes[selected[0]].content,
detail=notes[selected[0]].date)


# Funkcja dla przycisku "Modify selected" – włącza TRYB EDYCJI.
def edit_selected_item():
    # Pobieramy zaznaczony indeks.
    selected = note_list.curselection()
    # Jeśli coś jest zaznaczone...
    if selected:
     # Czyścimy pole "Title" – usuwamy tekst od znaku 0 do końca.
     note_title_entry.delete(0, tk.END)
     # Czyścimy pole "Content".
     note_content_entry.delete(0, tk.END)
     # Wpisujemy do pola "Title" (na pozycję 0) tytuł wybranej notatki.
     note_title_entry.insert(0, notes[selected[0]].title)
     # Wpisujemy do pola "Content" treść wybranej notatki.
     note_content_entry.insert(0, notes[selected[0]].content)
     # UKRYWAMY ramkę z 6 głównymi przyciskami (widżet dalej istnieje).
     button_frame.pack_forget()
     # POKAZUJEMY ramkę z przyciskami Confirm / Cancel:
```

```
     #   fill="x" -> rozciągnięta na całą szerokość
     #   pady=5   -> margines 5 px u góry i u dołu
     #   padx=20  -> margines 20 px z lewej i z prawej
     button_edit_frame.pack(fill="x", pady=5, padx=20)


# Funkcja dla przycisku "Confirm" – ZAPISUJE zmiany z edycji.
def confirm_edit():
    # Pobieramy zaznaczony indeks.
    selected = note_list.curselection()
    # Jeśli coś jest zaznaczone...
    if selected:
     # Pobieramy nowy tytuł z pola "Title" (tu bez .strip()!).
     title = note_title_entry.get()
     # Pobieramy nową treść z pola "Content".
     content = note_content_entry.get()
     # Jeśli tytuł nie jest pusty...
     if title:
         # Zmieniamy tytuł w obiekcie Note (data zostaje stara).
         notes[selected[0]].title = title
         # Zmieniamy treść w obiekcie Note.
         notes[selected[0]].content = content
         # Listbox nie ma "edytuj", więc USUWAMY stary tytuł...
         note_list.delete(selected[0])
         # ...i WSTAWIAMY nowy w to samo miejsce (ten sam indeks).
         note_list.insert(selected[0], title)
         # Ponownie zaznaczamy ten element na liście.
         note_list.select_set(selected[0])
         # Ukrywamy ramkę z przyciskami Confirm / Cancel.
         button_edit_frame.pack_forget()
         # Przywracamy ramkę z głównymi przyciskami
         # (fill="x", margines 5 px góra/dół, 20 px lewo/prawo).
         button_frame.pack(fill="x", pady=5, padx=20)
         # Czyścimy pole "Title".
         note_title_entry.delete(0, tk.END)
         # Czyścimy pole "Content".
         note_content_entry.delete(0, tk.END)


# Funkcja dla przycisku "Cancel" – ANULUJE edycję (nic nie zapisuje).
def cancel_edit():
    # Ukrywamy ramkę z przyciskami Confirm / Cancel.
    button_edit_frame.pack_forget()
    # Przywracamy ramkę z głównymi przyciskami
    # (fill="x", margines 5 px góra/dół, 20 px lewo/prawo).
    button_frame.pack(fill="x", pady=5, padx=20)
    # Czyścimy pole "Title".
    note_title_entry.delete(0, tk.END)
    # Czyścimy pole "Content".
    note_content_entry.delete(0, tk.END)


# ---------------------------------------------------------------------
#  OKNO GŁÓWNE
```

```
# ---------------------------------------------------------------------

# Tworzymy GŁÓWNE OKNO aplikacji (w programie może być tylko jedno Tk()).
window = tk.Tk()
# Ustawiamy tekst na pasku tytułu okna: "Notes app".
window.title("Notes app")
# Ustawiamy rozmiar okna: 350 px szerokości x 550 px wysokości.
window.geometry('350x550')
# Zmieniamy kolor tła okna na ciemnoszary (kod szesnastkowy #2C2C2C).
window.configure(background="#2C2C2C")


# ---------------------------------------------------------------------
#  NAGŁÓWEK "Notes"
# ---------------------------------------------------------------------

# Etykieta (ttk.Label) z napisem "Notes":
#   window                  -> rodzic: leży w głównym oknie
#   text="Notes"            -> wyświetlany tekst
#   foreground="orange red"  -> kolor tekstu: pomarańczowo-czerwony
#   background="#2C2C2C"     -> tło jak okno (ciemnoszare)
#   font=("Arial", 20, "bold") -> czcionka Arial, rozmiar 20, pogrubiona
label = ttk.Label(window, text="Notes", foreground="orange red", background="#2C2C2C",
font=("Arial", 20, "bold"))
# Umieszczamy etykietę metodą pack (na górze, wyśrodkowana):
#   pady=5 -> margines 5 px u góry i 5 px u dołu
label.pack(pady=5)


# ---------------------------------------------------------------------
#  LISTA NOTATEK (Listbox)
# ---------------------------------------------------------------------

# Listbox – lista, z której można zaznaczać elementy:
#   window                          -> rodzic: główne okno
#   font=("SegoeUI", 14, "bold")    -> czcionka SegoeUI, rozmiar 14, pogrubiona
#   bg="#414141"                    -> tło listy: szare (bg = background)
#   fg="white"                      -> kolor tekstu: biały (fg = foreground)
#   borderwidth=0                   -> brak obramowania
#   highlightthickness=0            -> brak ramki podświetlenia przy fokusie
note_list = tk.Listbox(window, font=("SegoeUI", 14, "bold"), bg="#414141", fg="white",
borderwidth=0, highlightthickness=0)
# Pętla for: dla każdej notatki (obiektu Note) z listy notes...
for note in notes:
    # ...dopisujemy jej tytuł na koniec Listboxa.
    note_list.insert(tk.END, note.title)
# Umieszczamy Listbox metodą pack:
#   fill="x"      -> rozciągnięty na całą szerokość
#   pady=(0, 5)   -> margines 0 px u góry i 5 px u dołu
#   padx=20       -> margines 20 px z lewej i z prawej
note_list.pack(fill="x", pady=(0,5), padx=20)
# Podpinamy zdarzenie: PODWÓJNE KLIKNIĘCIE LEWYM przyciskiem myszy
# ("<Double-Button-1>") wywołuje funkcję show_content(event).
note_list.bind("<Double-Button-1>", show_content)
```

```
# ---------------------------------------------------------------------
#  POLE "Title"
# ---------------------------------------------------------------------

# Etykieta z napisem "Title:":
#   window                      -> rodzic: główne okno
#   text="Title:"               -> wyświetlany tekst
#   font=("SegoeUI", 10, "bold")  -> czcionka SegoeUI, rozmiar 10, pogrubiona
#   background="#2C2C2C"        -> tło ciemnoszare (jak okno)
#   foreground="white"          -> biały tekst
note_title_label = ttk.Label(window, text="Title:", font=("SegoeUI", 10, "bold"),
background="#2C2C2C", foreground="white")
# Umieszczamy etykietę metodą pack:
#   anchor="w"    -> przyklejona do LEWEJ strony (w = west, zachód)
#   padx=20       -> margines 20 px z lewej i z prawej
#   pady=(5, 0)   -> margines 5 px u góry, 0 px u dołu (blisko pola)
note_title_label.pack(anchor="w", padx=20, pady=(5, 0))

# Pole tekstowe (ttk.Entry) na tytuł – jedna linia tekstu:
#   window                      -> rodzic: główne okno
#   font=("SegoeUI", 12, "bold")  -> czcionka SegoeUI, rozmiar 12, pogrubiona
note_title_entry = ttk.Entry(window, font=("SegoeUI", 12, "bold"))
# Umieszczamy pole metodą pack:
#   fill="x"  -> rozciągnięte na całą szerokość
#   pady=5    -> margines 5 px u góry i u dołu
#   padx=20   -> margines 20 px z lewej i z prawej
note_title_entry.pack(fill="x", pady=5, padx=20)


# ---------------------------------------------------------------------
#  POLE "Content"
# ---------------------------------------------------------------------

# Etykieta z napisem "Content:" – te same ustawienia co "Title:"
# (SegoeUI 10 pogrubiona, tło #2C2C2C, biały tekst).
note_content_label = ttk.Label(window, text="Content:", font=("SegoeUI", 10, "bold"),
background="#2C2C2C", foreground="white")
# pack: do lewej (anchor="w"), 20 px lewo/prawo, 5 px u góry, 0 px u dołu.
note_content_label.pack(anchor="w", padx=20, pady=(5, 0))

# Pole tekstowe na treść notatki (SegoeUI, 12, pogrubiona).
note_content_entry = ttk.Entry(window, font=("SegoeUI", 12, "bold"))
# pack: na całą szerokość (fill="x"), 5 px góra/dół, 20 px lewo/prawo.
note_content_entry.pack(fill="x", pady=5, padx=20)


# ---------------------------------------------------------------------
#  RAMKA Z 6 GŁÓWNYMI PRZYCISKAMI (siatka 3 wiersze x 2 kolumny)
# ---------------------------------------------------------------------

# Ramka (tk.Frame) – kontener na przyciski:
#   window                -> rodzic: główne okno
```

```
#   borderwidth=0         -> brak obramowania
#   background="#414141"  -> szare tło ramki
button_frame = tk.Frame(window, borderwidth=0, background="#414141")
# Konfiguracja kolumny 0 siatki w ramce:
#   weight=1            -> kolumna rozciąga się i dzieli wolne miejsce
#   uniform="buttons"   -> wszystkie kolumny z grupy "buttons" mają RÓWNĄ szerokość
button_frame.grid_columnconfigure(0, weight=1, uniform="buttons")
# Konfiguracja kolumny 1 – identycznie (ta sama grupa "buttons").
button_frame.grid_columnconfigure(1, weight=1, uniform="buttons")
# Umieszczamy ramkę w oknie metodą pack:
#   fill="x" -> na całą szerokość, pady=5 -> 5 px góra/dół, padx=20 -> 20 px lewo/prawo
# (W oknie używamy pack, a WEWNĄTRZ ramki grid – to poprawne,
#  bo nie mieszamy menedżerów w tym samym kontenerze.)
button_frame.pack(fill="x", pady=5, padx=20)

# Przycisk "Add last":
#   button_frame        -> rodzic: leży w ramce button_frame
#   text="Add last"     -> napis na przycisku
#   font=button_font    -> czcionka Helvetica 9 pogrubiona
#   bg="lightgreen"     -> jasnozielone tło
#   command=add_last    -> po kliknięciu wywołuje funkcję add_last (BEZ nawiasów!)
button_add = tk.Button(button_frame, text="Add last", font=button_font, bg="lightgreen",
command=add_last)
# Umieszczamy przycisk w siatce:
#   row=0, column=0  -> wiersz 0, kolumna 0 (lewy górny róg)
#   padx=5, pady=5   -> margines 5 px z każdej strony
#   sticky="ew"      -> rozciągnięty w poziomie (e = east/prawo, w = west/lewo)
button_add.grid(row=0, column=0, padx=5, pady=5, sticky="ew")

# Przycisk "Add first":
#   rodzic button_frame, napis "Add first",
#   fg="white"          -> biały tekst
#   font=button_font    -> Helvetica 9 pogrubiona
#   bg="green"          -> zielone tło
#   command=add_first   -> po kliknięciu wywołuje add_first
button_add_first = tk.Button(button_frame, text="Add first", fg="white", font=button_font,
bg="green", command=add_first)
# Siatka: wiersz 0, kolumna 1 (prawy górny róg), padding x 5 px, padding y 5 px,
# rozciągnięty w poziomie (sticky="ew").
button_add_first.grid(row=0, column=1, padx=5, pady=5, sticky="ew")

# Przycisk "Delete last":
#   rodzic button_frame, napis "Delete last",
#   bg="orange"           -> pomarańczowe tło
#   font=button_font      -> Helvetica 9 pogrubiona
#   command=delete_last   -> po kliknięciu wywołuje delete_last
button_delete = tk.Button(button_frame, text="Delete last", bg="orange", font=button_font,
command=delete_last)
# Siatka: wiersz 1, kolumna 0 (środek, lewa), padding x 5 px, padding y 5 px,
# rozciągnięty w poziomie.
button_delete.grid(row=1, column=0, padx=5, pady=5, sticky="ew")

# Przycisk "Delete all":
#   rodzic button_frame, napis "Delete all",
```

```
#   bg="red3"           -> ciemnoczerwone tło
#   fg="white"          -> biały tekst
#   font=button_font    -> Helvetica 9 pogrubiona
#   command=delete_all  -> po kliknięciu wywołuje delete_all
button_delete_all = tk.Button(button_frame, text="Delete all", bg="red3", fg="white",
font=button_font, command=delete_all)
# Siatka: wiersz 1, kolumna 1 (środek, prawa), padding x 5 px, padding y 5 px,
# rozciągnięty w poziomie.
button_delete_all.grid(row=1, column=1, padx=5, pady=5, sticky="ew")

# Przycisk "Delete selected":
#   rodzic button_frame, napis "Delete selected",
#   bg="orange red"               -> pomarańczowo-czerwone tło
#   font=button_font              -> Helvetica 9 pogrubiona
#   command=delete_selected_item  -> po kliknięciu wywołuje delete_selected_item
button_delete_selected = tk.Button(button_frame, text="Delete selected", bg="orange red",
font=button_font, command=delete_selected_item)
# Siatka: wiersz 2, kolumna 0 (lewy dolny róg), padding x 5 px, padding y 5 px,
# rozciągnięty w poziomie.
button_delete_selected.grid(row=2, column=0, padx=5, pady=5, sticky="ew")

# Przycisk "Modify selected":
#   rodzic button_frame,
#   background="cyan2"          -> turkusowe tło (pełna nazwa zamiast bg)
#   font=button_font            -> Helvetica 9 pogrubiona
#   text="Modify selected"      -> napis na przycisku
#   command=edit_selected_item  -> po kliknięciu wywołuje edit_selected_item
button_edit_selected = tk.Button(button_frame, background="cyan2", font=button_font,
text="Modify selected", command=edit_selected_item)
# Siatka: wiersz 2, kolumna 1 (prawy dolny róg), padding x 5 px, padding y 5 px,
# rozciągnięty w poziomie.
button_edit_selected.grid(row=2, column=1, padx=5, pady=5, sticky="ew")


# ---------------------------------------------------------------------
#  RAMKA EDYCJI (Confirm / Cancel) — NA START UKRYTA
# ---------------------------------------------------------------------

# Druga ramka: rodzic window, bez obramowania, szare tło #414141.
button_edit_frame = tk.Frame(window, borderwidth=0, background="#414141")
# Kolumna 0: rozciąga się (weight=1), grupa równej szerokości "edit_buttons".
button_edit_frame.grid_columnconfigure(0, weight=1, uniform="edit_buttons")
# Kolumna 1: tak samo — obie kolumny będą identycznej szerokości.
button_edit_frame.grid_columnconfigure(1, weight=1, uniform="edit_buttons")
# UWAGA: brak button_edit_frame.pack() -> ramka jest NIEWIDOCZNA,
# dopóki nie pokaże jej funkcja edit_selected_item.

# Przycisk "Confirm":
#   button_edit_frame     -> rodzic: ramka edycji
#   bg="lightgreen"       -> jasnozielone tło
#   text="Confirm"        -> napis na przycisku
#   font=button_font      -> Helvetica 9 pogrubiona
#   command=confirm_edit  -> po kliknięciu wywołuje confirm_edit
button_confirm = tk.Button(button_edit_frame, bg="lightgreen", text="Confirm",
```

```
font=button_font, command=confirm_edit)
# Siatka: wiersz 0, kolumna 0 (lewa), padding x 5 px, padding y 5 px,
# rozciągnięty w poziomie.
button_confirm.grid(row=0, column=0, padx=5, pady=5, sticky="ew")

# Przycisk "Cancel":
#   rodzic button_edit_frame,
#   bg="red3"            -> ciemnoczerwone tło
#   text="Cancel"        -> napis na przycisku
#   fg="white"           -> biały tekst
#   font=button_font     -> Helvetica 9 pogrubiona
#   command=cancel_edit  -> po kliknięciu wywołuje cancel_edit
button_cancel = tk.Button(button_edit_frame, bg="red3", text="Cancel", fg="white",
font=button_font, command=cancel_edit)
# Siatka: wiersz 0, kolumna 1 (prawa), padding x 5 px, padding y 5 px,
# rozciągnięty w poziomie.
button_cancel.grid(row=0, column=1, padx=5, pady=5, sticky="ew")


# ---------------------------------------------------------------------
#  START PROGRAMU
# ---------------------------------------------------------------------

# Uruchamiamy PĘTLĘ ZDARZEŃ – okno się pokazuje i czeka na kliknięcia,
# wpisywanie itd. Program "stoi" w tej linijce aż do zamknięcia okna.
window.mainloop()
```

Część C2 – kod blok po bloku z dokładnym objaśnieniem

1. Importy

```
import tkinter as tk
from tkinter import ttk, messagebox
from datetime import datetime
```

- Linia 1: importuje tkinter pod skrótem `tk`. Dalej piszemy `tk.Tk()`, `tk.Button` itd.
- Linia 2: z pakietu tkinter bierze moduł `ttk` (ładniejsze widżety) i `messagebox` (okienka dialogowe).
- Linia 3: z modułu `datetime` importuje klasę `datetime`, potrzebną do pobrania aktualnej daty.

2. Klasa Note

```
class Note:
    def __init__(self, title="", content=""):
     self.title = title
```

```
        self.content = content
        self.date = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
```

- `class Note:` to klasa opisująca jedną notatkę.
- `__init__` to konstruktor, uruchamia się przy `Note(...)`.
- `self`: tworzony obiekt.
- `title=""`, `content=""`: domyślne wartości, gdy nic nie podamy.
- `self.title = title`: zapisuje tytuł w obiekcie.
- `self.content = content`: zapisuje treść.
- `self.date = ...`: zapisuje **datę utworzenia** notatki jako napis, np. `"29/10/2026 21:43:05"`.

Przykład użycia:

```
n = Note("Zakupy", "mleko, chleb")
print(n.title)    # Zakupy
print(n.content)  # mleko, chleb
print(n.date)     # 29/10/2026 21:43:05
```

3. Zmienne globalne

```
button_font = ("Helvetica", 9, "bold")
notes = []
notes.append(Note("Pierwsza notatka", "Treść pierwszej notatki"))
```

- `button_font`: krotka z czcionką używaną we wszystkich przyciskach (żeby nie powtarzać).
- `notes = []`: pusta lista, która przechowuje **obiekty** `Note`, czyli dane programu.
- `notes.append(...)`: na start dodaje jedną przykładową notatkę.

Ważne: program trzyma dane w dwóch miejscach, które muszą być zgodne:

- `notes`: lista **obiektów** (tytuł, treść, data);
- `note_list` (Listbox): to, co **widać** na ekranie (same tytuły).

Indeks w `notes` odpowiada indeksowi w `note_list`. Dlatego każda funkcja zmienia **oba** miejsca.

4. Funkcja `add_last` – dodaj na koniec

```
def add_last():
    title = note_title_entry.get().strip()
    if title:
        content = note_content_entry.get()
        notes.append(Note(title, content))
        note_list.insert(tk.END, title)
```

- Pobiera tytuł z pola, `.strip()` usuwa spacje z brzegów.
- `if title:`: jeśli tytuł nie jest pusty (sam tytuł ze spacji też odpadnie dzięki `strip`).
- Pobiera treść z drugiego pola.
- `notes.append(...)`: tworzy nowy obiekt `Note` i dodaje na koniec listy danych.
- `note_list.insert(tk.END, title)`: dodaje tytuł na koniec listy na ekranie.

5. Funkcja `add_first` – dodaj na początek

```
def add_first():
    title = note_title_entry.get().strip()
    if title:
     content = note_content_entry.get()
     notes.insert(0, Note(title, content))
     note_list.insert(0, title)
```

Tak samo jak wyżej, ale `insert(0, ...)` wstawia na **pozycję 0**, czyli na początek, zarówno w liście danych, jak i w Listboxie.

6. Funkcja `delete_last` – usuń ostatnią

```
def delete_last():
    notes.pop()
    note_list.delete(tk.END, tk.END)
```

- `notes.pop()`: usuwa ostatni obiekt z listy.
- `note_list.delete(tk.END, tk.END)`: usuwa ostatni element z Listboxa (od końca do końca).
- Brak `if`: przy pustej liście `pop()` rzuci błąd `IndexError` (patrz część D).

7. Funkcja `delete_all` – usuń wszystko + dymek

```
def delete_all():
    notes.clear()
    note_list.delete(0, tk.END)
    label_toast = tk.Label(background="black", foreground="white", font=button_font,
text="Deleted")
    label_toast.place(relx=0.5, rely=0.45, anchor="center")
    label_toast.after(3000, label_toast.destroy)
```

- `notes.clear()`: czyści listę danych.
- `note_list.delete(0, tk.END)`: usuwa elementy od 0 do końca, czyli wszystkie.
- Tworzy etykietę „Deleted” (czarne tło, biały tekst). Nie podano rodzica, więc trafia do głównego okna.
- `place(relx=0.5, rely=0.45, anchor="center")`: wyświetla ją na środku w poziomie, na 45% wysokości, nad innymi widżetami. Takie powiadomienie nazywa się „toast”.
- `after(3000, label_toast.destroy)`: po 3 sekundach usuwa etykietę. `destroy` jest **bez nawiasów**, bo przekazujemy funkcję, a nie jej wynik.

8. Funkcja `delete_selected_item` – usuń zaznaczoną

```
def delete_selected_item():
    selected = note_list.curselection()
    if selected:
     notes.pop(selected[0])
     note_list.delete(selected)
```

- `curselection()` zwraca krotkę, np. `(2,)`, albo `()` gdy nic nie zaznaczono.
- `if selected:`: działa tylko, gdy coś jest zaznaczone.
- `notes.pop(selected[0])`: usuwa z danych element o tym indeksie.
- `note_list.delete(selected)`: usuwa z Listboxa (tkinter przyjmie krotkę `(2,)` jak indeks 2; czytelniej byłoby `delete(selected[0])`).

9. Funkcja `show_content` – podgląd po podwójnym kliknięciu

```
def show_content(event):
    selected = note_list.curselection()
    messagebox.showinfo(notes[selected[0]].title, notes[selected[0]].content,
detail=notes[selected[0]].date)
```

- Ma parametr `event`, bo jest podpięta przez `bind` (tkinter przekazuje obiekt zdarzenia).
- Pobiera zaznaczony indeks.
- `messagebox.showinfo(tytuł, wiadomość, detail=...)`: okienko z tytułem notatki na pasku, treścią w środku i datą jako dodatkowym tekstem.

10. Funkcja `edit_selected_item` – tryb edycji

```
def edit_selected_item():
    selected = note_list.curselection()
    if selected:
     note_title_entry.delete(0, tk.END)
     note_content_entry.delete(0, tk.END)
     note_title_entry.insert(0, notes[selected[0]].title)
     note_content_entry.insert(0, notes[selected[0]].content)
     button_frame.pack_forget()
     button_edit_frame.pack(fill="x", pady=5, padx=20)
```

- Jeśli coś jest zaznaczone:
  - czyści oba pola tekstowe (`delete(0, tk.END)`);
  - wpisuje do nich tytuł i treść wybranej notatki (`insert(0, ...)`);
  - `button_frame.pack_forget()` **chowa** ramkę ze zwykłymi przyciskami;
  - `button_edit_frame.pack(...)` **pokazuje** ramkę z przyciskami Confirm i Cancel.

To jest przełączanie „ekranów” przez ukrywanie i pokazywanie ramek.

11. Funkcja `confirm_edit` – zatwierdź zmiany

```
def confirm_edit():
    selected = note_list.curselection()
    if selected:
     title = note_title_entry.get()
     content = note_content_entry.get()
     if title:
         notes[selected[0]].title = title
         notes[selected[0]].content = content
         note_list.delete(selected[0])
         note_list.insert(selected[0], title)
         note_list.select_set(selected[0])
```

```
        button_edit_frame.pack_forget()
        button_frame.pack(fill="x", pady=5, padx=20)
        note_title_entry.delete(0, tk.END)
        note_content_entry.delete(0, tk.END)
```

- Pobiera zaznaczony indeks i nowe wartości z pól.
- Jeśli tytuł nie jest pusty:
    - zmienia atrybuty istniejącego obiektu (`.title`, `.content`). **Data się nie zmienia**;
    - w Listboxie nie ma „edytuj”, więc **usuwa stary element i wstawia nowy w to samo miejsce**;
    - `select_set(...)` ponownie zaznacza ten element;
    - chowa ramkę edycji, pokazuje z powrotem zwykłe przyciski;
    - czyści pola tekstowe.

12. Funkcja `cancel_edit` – anuluj edycję

```
def cancel_edit():
    button_edit_frame.pack_forget()
    button_frame.pack(fill="x", pady=5, padx=20)
    note_title_entry.delete(0, tk.END)
    note_content_entry.delete(0, tk.END)
```

Nic nie zmienia w danych: chowa przyciski edycji, przywraca normalne i czyści pola.

13. Okno główne

```
window = tk.Tk()
window.title("Notes app")
window.geometry('350x550')
window.configure(background="#2C2C2C")
```

- Tworzy główne okno.
- Tytuł na pasku: „Notes app”.
- Rozmiar 350 px szerokości na 550 px wysokości.
- Tło ciemnoszare (hex `#2C2C2C`).

Uwaga: funkcje z góry pliku używają zmiennych `note_list`, `note_title_entry` itd., które powstają dopiero tutaj. To działa, bo funkcje są **wywoływane później** (po kliknięciu), a nie w momencie definicji.

14. Nagłówek i lista notatek

```
label = ttk.Label(window, text="Notes", foreground="orange red", background="#2C2C2C",
font=("Arial", 20, "bold"))
label.pack(pady=5)
note_list = tk.Listbox(window, font=("SegoeUI", 14, "bold"), bg="#414141", fg="white",
borderwidth=0, highlightthickness=0)
for note in notes:
    note_list.insert(tk.END, note.title)
```

```
note_list.pack(fill="x", pady=(0,5), padx=20)
note_list.bind("<Double-Button-1>", show_content)
```

- Etykieta „Notes”: pomarańczowo-czerwona, Arial 20 pogrubiony, tło jak okno.
- `label.pack(pady=5)`: umieszcza ją na górze z marginesem 5 px u góry i u dołu.
- `tk.Listbox(...)`: lista z ciemnym tłem (`bg`), białym tekstem (`fg`), bez obramowania i bez ramki fokusu.
- Pętla `for`: wpisuje do Listboxa tytuły wszystkich notatek, które już są w `notes` (na starcie: „Pierwsza notatka”).
- `pack(fill="x", pady=(0,5), padx=20)`: rozciąga listę na szerokość, 0 px marginesu u góry, 5 px u dołu, 20 px po bokach.
- `bind("<Double-Button-1>", show_content)`: podwójne kliknięcie lewym przyciskiem wywołuje `show_content(event)`.

15. Pola Title i Content

```
note_title_label = ttk.Label(window, text="Title:", font=("SegoeUI", 10, "bold"),
background="#2C2C2C", foreground="white")
note_title_label.pack(anchor="w", padx=20, pady=(5, 0))

note_title_entry = ttk.Entry(window, font=("SegoeUI", 12, "bold"))
note_title_entry.pack(fill="x", pady=5, padx=20)

note_content_label = ttk.Label(window, text="Content:", font=("SegoeUI", 10, "bold"),
background="#2C2C2C", foreground="white")
note_content_label.pack(anchor="w", padx=20, pady=(5, 0))

note_content_entry = ttk.Entry(window, font=("SegoeUI", 12, "bold"))
note_content_entry.pack(fill="x", pady=5, padx=20)
```

- Etykieta „Title:”: `anchor="w"` przykleja ją do **lewej**, `pady=(5, 0)` daje 5 px u góry i 0 u dołu (żeby była blisko pola).
- `ttk.Entry`: jednoliniowe pole tekstowe na tytuł, rozciągnięte na szerokość (`fill="x"`).
- To samo dla „Content:” i pola treści.

16. Ramka z głównymi przyciskami

```
button_frame = tk.Frame(window, borderwidth=0, background="#414141")
button_frame.grid_columnconfigure(0, weight=1, uniform="buttons")
button_frame.grid_columnconfigure(1, weight=1, uniform="buttons")
button_frame.pack(fill="x", pady=5, padx=20)
```

- `tk.Frame`: ramka (kontener) na przyciski, z tłem `#414141`.
- `grid_columnconfigure(0, ...)` i `(1, ...)`: dwie kolumny siatki; `weight=1` sprawia, że dzielą wolne miejsce, a `uniform="buttons"` że mają **identyczną szerokość**.
- Sama ramka jest spakowana `pack` w oknie, a w jej środku używamy `grid`. To poprawne, bo to różne kontenery.

17. Przyciski (siatka 3 wiersze x 2 kolumny)

```
button_add = tk.Button(button_frame, text="Add last", font=button_font, bg="lightgreen",
command=add_last)
button_add.grid(row=0, column=0, padx=5, pady=5, sticky="ew")

button_add_first = tk.Button(button_frame, text="Add first", fg="white", font=button_font,
bg="green", command=add_first)
button_add_first.grid(row=0, column=1, padx=5, pady=5, sticky="ew")

button_delete = tk.Button(button_frame, text="Delete last", bg="orange", font=button_font,
command=delete_last)
button_delete.grid(row=1, column=0, padx=5, pady=5, sticky="ew")

button_delete_all = tk.Button(button_frame, text="Delete all", bg="red3", fg="white",
font=button_font, command=delete_all)
button_delete_all.grid(row=1, column=1, padx=5, pady=5, sticky="ew")

button_delete_selected = tk.Button(button_frame, text="Delete selected", bg="orange red",
font=button_font, command=delete_selected_item)
button_delete_selected.grid(row=2, column=0, padx=5, pady=5, sticky="ew")

button_edit_selected = tk.Button(button_frame, background="cyan2", font=button_font,
text="Modify selected", command=edit_selected_item)
button_edit_selected.grid(row=2, column=1, padx=5, pady=5, sticky="ew")
```

Układ:

	kolumna 0	**kolumna 1**
wiersz 0	Add last → `add_last`	Add first → `add_first`
wiersz 1	Delete last → `delete_last`	Delete all → `delete_all`
wiersz 2	Delete selected → `delete_selected_item`	Modify selected → `edit_selected_item`

- Rodzicem każdego przycisku jest `button_frame`, więc przyciski są w ramce.
- `command=funkcja`: co się dzieje po kliknięciu (bez nawiasów!).
- `sticky="ew"`: przycisk rozciąga się na całą szerokość komórki.
- `padx=5, pady=5`: odstępy między przyciskami.

18. Ramka edycji (na start ukryta)

```
button_edit_frame = tk.Frame(window, borderwidth=0, background="#414141")
button_edit_frame.grid_columnconfigure(0, weight=1, uniform="edit_buttons")
button_edit_frame.grid_columnconfigure(1, weight=1, uniform="edit_buttons")

button_confirm = tk.Button(button_edit_frame, bg="lightgreen", text="Confirm",
font=button_font, command=confirm_edit)
button_confirm.grid(row=0, column=0, padx=5, pady=5, sticky="ew")

button_cancel = tk.Button(button_edit_frame, bg="red3", text="Cancel", fg="white",
```

```
font=button_font, command=cancel_edit)
button_cancel.grid(row=0, column=1, padx=5, pady=5, sticky="ew")
```

- Druga ramka z przyciskami Confirm i Cancel, ułożonymi tak samo (2 równe kolumny).
- **Nie ma tu** `button_edit_frame.pack()`, więc na starcie ramka jest **niewidoczna**. Pokazuje ją dopiero `edit_selected_item`.

19. Start programu

```
window.mainloop()
```

Uruchamia pętlę zdarzeń: okno się pokazuje i reaguje na kliknięcia aż do zamknięcia.

Przepływ działania

1. Start: okno z listą („Pierwsza notatka”), dwoma polami i 6 przyciskami.
2. Wpisujesz tytuł i treść, potem **Add last** albo **Add first**, a notatka pojawia się na liście.
3. **Podwójne kliknięcie** na notatce otwiera okienko z treścią i datą.
4. Zaznaczasz notatkę i klikasz **Modify selected**: pola się wypełniają, przyciski zmieniają się na Confirm/Cancel.
5. **Confirm** zapisuje zmiany, **Cancel** porzuca. Wracają zwykłe przyciski.
6. **Delete all** czyści wszystko i na 3 s pokazuje dymek „Deleted”.

Część D – pułapki i błędy w programie (mogą być pytania „co się stanie, gdy…”)

1. `delete_last` **na pustej liście** daje `IndexError: pop from empty list` (brak `if notes:`). Poprawka:

```
def delete_last():
    if notes:
     notes.pop()
     note_list.delete(tk.END)
```

2. `show_content` **bez zaznaczenia**: `selected` to `()`, więc `selected[0]` daje `IndexError` (np. podwójne kliknięcie w puste miejsce listy). Poprawka: dodać `if selected:`.

3. Zgubione zaznaczenie przy edycji: domyślnie Listbox ma `exportselection=True`, więc zaznaczenie tekstu w polu Entry może skasować zaznaczenie na liście. Wtedy `confirm_edit` nic nie zrobi, bo `curselection()` jest puste. Poprawka: `tk.Listbox(..., exportselection=False)` albo zapamiętanie indeksu w zmiennej w `edit_selected_item`.

4. `confirm_edit` **nie robi** `.strip()`, więc tytuł z samych spacji przejdzie (w odróżnieniu od dodawania).

5. Data nie zmienia się po edycji: zostaje data utworzenia.

6. `"SegoeUI"`: prawdziwa nazwa czcionki to `"Segoe UI"` (ze spacją). Gdy czcionki nie ma, tkinter użyje domyślnej, bez błędu.

7. `command=add_last()` **(z nawiasami)** to częsty błąd: funkcja wykonałaby się raz przy tworzeniu przycisku, a `command` dostałby `None`.

Część E – ściąga: pack / grid / place

```
# pack – jeden pod drugim
w.pack()                          # na górę, wyśrodkowany
w.pack(side="left")               # obok siebie od lewej (side: top/bottom/left/right)
w.pack(fill="x")                  # rozciągnij w poziomie (x / y / both)
w.pack(expand=True, fill="both") # zajmij całe wolne miejsce
w.pack(padx=10, pady=(5, 0))      # marginesy (lewo/prawo, góra/dół)
w.pack(anchor="w")                # przyklej do lewej
w.pack_forget()                   # ukryj

# grid – tabela
w.grid(row=0, column=0)
w.grid(row=1, column=0, columnspan=2)  # zajmij 2 kolumny
w.grid(sticky="ew")                    # rozciągnij poziomo (n, s, e, w, nsew)
frame.grid_columnconfigure(0, weight=1)
w.grid_forget()

# place – dokładna pozycja
w.place(x=10, y=20)                          # w pikselach
w.place(relx=0.5, rely=0.5, anchor="center") # względnie (0–1)
w.place_forget()
```

Minimalny program w tkinterze (warto znać na pamięć)

```
import tkinter as tk

def klik():
    etykieta.config(text="Kliknięto!")

okno = tk.Tk()
okno.title("Test")
okno.geometry("300x200")

etykieta = tk.Label(okno, text="Witaj")
etykieta.pack()

przycisk = tk.Button(okno, text="Kliknij", command=klik)
przycisk.pack(pady=10)

okno.mainloop()
```

Część F – przykładowe pytania na sprawdzian

Pytania zamknięte (odpowiedzi na końcu)

1. Co robi metoda `__init__` w klasie? a) usuwa obiekt b) jest konstruktorem, wywołuje się przy tworzeniu obiektu c) uruchamia okno d) importuje moduł
2. Co oznacza `self` w metodzie klasy? a) klasę b) moduł c) konkretny obiekt, na którym wywołano metodę d) okno główne
3. Co robi `window.mainloop()`? a) zamyka okno b) uruchamia pętlę zdarzeń i wyświetla okno c) tworzy okno d) odświeża Listbox
4. Co zwraca `note_list.curselection()`, gdy nic nie jest zaznaczone? a) `None` b) `0` c) `-1` d) pustą krotkę `()`
5. Który zapis jest poprawny? a) `tk.Button(okno, command=funkcja())` b) `tk.Button(okno, command=funkcja)` c) `tk.Button(okno, command="funkcja")` d) `tk.Button(okno, onclick=funkcja)`
6. Co oznacza `fill="x"` w `pack()`? a) rozciągnij w pionie b) rozciągnij w poziomie c) ukryj widżet d) przesuń w prawo
7. Co robi `pack_forget()`? a) niszczy widżet na zawsze b) ukrywa widżet, który można potem znów pokazać c) czyści pole tekstowe d) usuwa listę
8. Co oznacza `"%M"` w `strftime`? a) miesiąc b) minuty c) milisekundy d) poniedziałek
9. Co robi `label_toast.after(3000, label_toast.destroy)`? a) niszczy etykietę od razu b) czeka 3 s, blokując program c) po 3000 ms usuwa etykietę, nie blokując programu d) przesuwa etykietę o 3000 px
10. Co oznacza `sticky="ew"` w `grid()`? a) widżet rozciąga się w poziomie (lewo–prawo) b) w pionie c) przykleja się do góry d) ukrywa się
11. Jakie zdarzenie to `"<Double-Button-1>"`? a) dwa klawisze b) podwójne kliknięcie lewym przyciskiem myszy c) prawy przycisk d) Enter
12. Co robi `" tekst ".strip()`? a) `"tekst"` b) `" tekst"` c) `"tekst "` d) błąd

Odpowiedzi: 1b, 2c, 3b, 4d, 5b, 6b, 7b, 8b, 9c, 10a, 11b, 12a

Pytania otwarte (z przykładowymi odpowiedziami)

1. Opisz, do czego służy klasa `Note` i jakie ma atrybuty. Klasa `Note` reprezentuje jedną notatkę. Konstruktor `__init__` przyjmuje tytuł i treść (domyślnie puste) i zapisuje je w atrybutach `title` i `content`. Atrybut `date` zawiera datę i godzinę utworzenia w formacie `dd/mm/rrrr gg:mm:ss`.

2. Napisz linijkę, która tworzy obiekt `Note` z tytułem „Zakupy” i treścią „mleko” i dodaje go na koniec listy `notes`.

```
notes.append(Note("Zakupy", "mleko"))
```

3. Napisz kod, który tworzy okno o tytule „Moja aplikacja” i rozmiarze 400x300.

```
okno = tk.Tk()
okno.title("Moja aplikacja")
okno.geometry("400x300")
okno.mainloop()
```

4. Napisz linijkę tworzącą przycisk „Zapisz” w oknie `window`, który wywołuje funkcję `zapisz`, i umieść go za pomocą `pack`.

```
tk.Button(window, text="Zapisz", command=zapisz).pack()
```

5. Jak wyczyścić pole tekstowe `note_title_entry`?

```
note_title_entry.delete(0, tk.END)
```

6. Jak usunąć wszystkie elementy z Listboxa `note_list`?

```
note_list.delete(0, tk.END)
```

7. Jak pobrać tekst z pola Entry i usunąć z niego spacje z brzegów?

```
tekst = note_title_entry.get().strip()
```

8. Wyjaśnij różnicę między `pack()`, `grid()` i `place()`. `pack()` układa widżety jeden pod drugim (lub obok siebie, z `side`). `grid()` układa je w tabeli wierszy i kolumn. `place()` ustawia widżet w dokładnej pozycji (piksele lub wartości względne 0–1). W jednym kontenerze nie należy mieszać `pack` i `grid`.

9. Dlaczego funkcja `show_content` ma parametr `event`, a `add_last` nie ma? `show_content` jest podpięta przez `bind`, a tkinter przy zdarzeniu przekazuje do funkcji obiekt zdarzenia, więc funkcja musi go przyjąć. `add_last` jest podpięta przez `command=` przycisku, które wywołuje funkcję bez argumentów.

10. Opisz, jak działa tryb edycji w programie. Po kliknięciu „Modify selected” program pobiera zaznaczoną notatkę, wpisuje jej tytuł i treść do pól, ukrywa ramkę z głównymi przyciskami (`pack_forget`) i pokazuje ramkę z Confirm/Cancel (`pack`). Confirm zapisuje nowe wartości w obiekcie i podmienia tytuł w Listboxie (usuń i wstaw w to samo miejsce), Cancel tylko przywraca przyciski i czyści pola.

11. Co się stanie, gdy klikniesz „Delete last” przy pustej liście? Jak to poprawić? `notes.pop()` rzuci `IndexError: pop from empty list`. Poprawka: dodać warunek `if notes:` przed usuwaniem.

12. Napisz linijkę, która wyświetla okienko informacyjne z tytułem „Info” i tekstem „Zapisano”.

```
messagebox.showinfo("Info", "Zapisano")
```

13. Napisz kod, który podpina funkcję `pokaz` pod podwójne kliknięcie na `note_list`.

```
note_list.bind("<Double-Button-1>", pokaz)
```

14. Napisz linijkę, która usuwa etykietę `lbl` po 2 sekundach.

```
lbl.after(2000, lbl.destroy)
```

15. Napisz kod, który umieszcza przycisk `b` w drugim wierszu i pierwszej kolumnie siatki, rozciągnięty w poziomie.

```
b.grid(row=1, column=0, sticky="ew")
```

(wiersze i kolumny liczymy od 0)

16. Dlaczego program przechowuje dane w liście `notes` i osobno w Listboxie? Listbox przechowuje tylko tekst do wyświetlenia (tytuły). Pełne dane (tytuł, treść, data) są w obiektach `Note` w liście `notes`. Indeksy w obu muszą się zgadzać, dlatego każda operacja (dodaj, usuń, edytuj) zmienia oba miejsca.

17. Do czego służą `weight=1` i `uniform` w `grid_columnconfigure`? `weight=1` sprawia, że kolumna rozciąga się i dzieli wolne miejsce z innymi kolumnami. `uniform="nazwa"` sprawia, że wszystkie kolumny z tej samej grupy mają identyczną szerokość, niezależnie od długości tekstu na przyciskach.